

Thesis Proposal

Eric Giovannini

April 11, 2025

Contents

1	Introduction	1
1.1	Gradual Typing: Background	2
1.2	Prior Work	2
2	Gradual Typing for Effect Handlers	4
3	Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory	5
3.1	Guarded Type Theory	5
3.2	Result	5
3.3	Modeling Gradually-Typed Programs	6
3.4	Modeling Dynamic Typing	6
3.5	Relational Semantics	7
3.5.1	A First Attempt at a Relational Model	7
3.5.2	Adjusting the Model: Perturbations	8
3.6	Summary of the Model	8
3.7	Agda Formalization	9
4	Extending to More Advanced Language Features	9
5	Plan for Remainder of Thesis Work	10

1 Introduction

My thesis work is in the area of *semantics for gradually typed programming languages*. Roughly speaking, gradually-typed languages allow the programmer to combine both static and dynamic typing disciplines in the same codebase, and to *gradually* migrate smoothly from a dynamic to static style over time. My goal has been to advance the state of the art in the mathematical tools used to model gradually-typed languages and ensure that the languages satisfy the intended formal properties.

In the remainder of this section, we provide background on gradual typing and the desired properties of gradually-typed languages. We then discuss prior approaches to formally proving these criteria, with a view towards motivating the novel approach developed in my thesis work.

1.1 Gradual Typing: Background

In programming language design, there is a tension between *static* typing and *dynamic* typing disciplines. With static typing, the code is type-checked at compile time, while in dynamic typing, the type checking is deferred to run-time. Both approaches have benefits and excel in different scenarios, with static typing offering compile-time assurance of a program’s type safety and type-based reasoning principles that justify program optimizations, and dynamic typing allowing for rapid prototyping of a codebase without committing to fixed type signatures. Most languages choose between static or dynamic typing and as a result, programmers that initially write their code in a dynamically typed language need to rewrite some or all of their codebase in a static language if they would like to receive the benefits of static typing once their codebase has matured.

Gradually typed languages [19, 22] seek to resolve this tension by allowing for both static and dynamic typing disciplines to be used in the same codebase. These languages support smooth interoperability between statically-typed and dynamically-typed styles. As a result, the programmer can begin with fully dynamically-typed code and *gradually* add explicit typing information to migrate portions of the codebase to a statically typed style without needing to rewrite the entire project in a completely different language.

The idea of a program becoming “more static” over time as the programmer performs the migration is made formal via a *term precision ordering* on programs. Term precision gives us the ability to compare programs based on their relative amount of static typing information. We think of this as a syntactic relation on programs where a program M is more static or precise than N if, roughly speaking, M has more typing annotations than N .

One of the fundamental theorems for gradually typed languages is *graduality*, also known as the *dynamic gradual guarantee*, originally defined by Siek, Vitousek, Cimini, and Boyland [20, 12]. Informally, graduality says that migrating code from dynamic to static typing should only allow for the introduction of static or dynamic type errors, and not otherwise change the behavior of the program. This is a way to formalize the programmer’s intuition that increasing precision corresponds to a generalized form of runtime assertions: there are no observable behavioral changes up to the point of the first dynamic type error.

Another desired property of gradually typed languages is that they should offer some of the benefits of static typing. While standard type soundness, that well-typed programs are free from runtime errors, is not compatible with runtime type errors, it is possible instead to prove that gradually typed languages validate *type-based reasoning*. For instance, while dynamically typed λ calculi only satisfy β equality for their type formers, statically typed λ calculi additionally satisfy type-dependent η properties that ensure that functions are determined by their behavior under application and that pattern matching on data types is safe and exhaustive. A gradually typed calculus that validates these type-dependent η laws then provides some of the type-based reasoning that dynamic languages lack.

1.2 Prior Work

There are several methods of establishing the desired properties of gradually-typed languages; we discuss these below.

Siek et al., in the same work that introduces the dynamic gradual guarantee [20], also prove the property for a simple gradually-typed language via a syntactic simulation argument. This proof method is closely tied to the syntax of the language, and there is limited opportunity for reuse of common abstractions across different languages. This lack of reusability is addressed somewhat by Siek and Chen [18], who give a framework for

proving graduality in the Agda proof assistant. The framework is parameterized by the specific language for which one wants to prove graduality, and the proof of graduality proceeds via the syntactic simulation argument. However, the main lemmas of the work are still tied to the particular operational syntactic calculus being modeled.

A second line of work for proving graduality consists of starting with a statically-typed language, and *deriving* from it a language that satisfies graduality by construction. The methods of Abstracting Gradual Typing [7] and the formal tools of the Gradualizer [6] follow this approach. The main downside to these approaches lies in their inflexibility: since the process is entirely mechanical, the language designer must adhere to the predefined framework. Many gradually typed languages do not fit into either framework, e.g., Typed Racket [22, 23], and the semantics produced is not always the desired one. Furthermore, while these frameworks do prove graduality of the resulting languages, they do not show the correctness of the equational theory, which is equally important to sound gradual typing.

Another common approach to proving graduality involves defining a semantic model for a gradually-typed language and establishing the graduality property in the model. Providing a semantics for gradual typing is inherently complicated in that it involves (1) recursion and recursive types through the presence of dynamic typing, (2) effects in the form of divergence and errors, and (3) relational models to capture the graduality property. Any such semantics needs to address each of these points.

One major line of work in the semantic approach consists of modeling the language via an operational semantics and defining a step-indexed logical relation over that semantics [12, 17, 15, 13]. These techniques have been used to model a variety of languages combining gradual typing with other interesting features. Step-indexed techniques are robust enough to scale to essentially arbitrary self-referential definitions, which means they can model highly recursive programming language features such as higher-order store and runtime-extensible dynamic types [2, 1, 11, 15]. However, as is common with operational approaches, the graduality proofs based on these techniques are highly repetitive and technical. Each development formulates a new logical relation from first-principles and proves many of the same tedious lemmas without reusable mathematical abstractions across languages.

In the quest for a more compositional and language-independent approach, New and Licata [16] constructed a denotational semantics for gradual typing using the tools of classical domain theory (e.g., ω -CPOs). This model was extended by Lennon-Bertrand, Maillard, Tabareau and Tanter [10] to prove graduality of a gradual dependently typed calculus CastCIC^G . This domain-theoretic approach demonstrated that it is possible to give a compositional semantic framework for proving graduality. However, an approach based on classical domain theory has fundamental limitations: domain theory appears incapable of modeling certain perversely recursive features of programming languages such as dynamic type tag generation and higher-order references [5], which are commonplace in real-world gradually typed systems as well as gradual calculi.

The goal of the bulk of my thesis work has been to combine the benefits of both the denotational and step-indexed approaches — the compositional nature of the denotational approach and the expressive power of step-indexing. We therefore take as a starting point the denotational approach of New and Licata. However, rather than work with classical domain theory, we instead work with *guarded type theory*, a *synthetic* form of step-indexing. The synthetic approach to step-indexing introduces additional primitives to the type theory that capture the essence of step-indexing axiomatically. It is especially useful as a setting in which to define denotational semantics of languages involving recursive types, making it a well-suited candidate for modelling gradually-typed languages. Unfortunately, the compositional theory of New-Licata breaks down in the guarded setting. My thesis work provides a novel refinement to their semantics that allows for some

compositional reasoning to be recovered even in the guarded setting.

The claim put forth in this thesis is that **the use of synthetic methods facilitates the construction of compositional semantics for real-world gradually-typed programming languages**. In the remainder of this document, we substantiate this claim by describing the two major research projects that make up the core of my thesis work. The second project forms the majority of my thesis work so far and is more directly relevant to my claim, and we therefore devote significantly more space to discussing it. We then discuss the next major step in this research program and outline a plan for the work to be completed.

2 Gradual Typing for Effect Handlers

My work on gradual typing began with a paper investigating the combination of gradual typing with algebraic effect handlers [13]. In that paper, we presented a new gradually-typed language, called GrEff, with support for algebraic effects and handlers. We sought to study the question of retrofitting an effect typing system onto an existing language with effects that lacks such a system. In GrEff, the code in a program can exist along a spectrum between completely *fully unchecked* and *fully checked* effects. In the former, the type system is concerned only with the input and output types of effect operations. With the latter, the type system additionally tracks which effects a given part of the program may raise.

The motivation for this work was less about the approach to proving graduality for GrEff, and more about the design of the language itself. Indeed, the method by which we proved graduality for GrEff is an extension of the operational logical relations methods employed in prior work. There were some novelties related to handling gradual subtyping in the semantics, but these were largely straightforward.

However, while constructing these proofs, the lack of reusable building blocks afforded by the operational approach became especially apparent. Additionally, manual step-index manipulation was a pervasive part of the development and proved to be tedious and error-prone. Overall, the time and effort required to carry out the proofs was enormous: the appendix in the technical report is over 60 pages long [14]! We concluded that while the operational approach was sufficient for proving graduality, the need to reprove all the basic lemmas and essentially redo the entire development for each new language presents a significant roadblock to ensuring the soundness of future gradually-typed languages.

Of course, the operational approach based on step-indexing is not the only means of establishing graduality. As discussed above, the denotational model of New and Licata [16] is not tied to any particular operational syntactic calculus. Instead, their semantic framework consists of language-independent, compositional abstractions. However, by being based on classical domain theory, the semantics of New and Licata is limited in what language features it can handle. Specifically, it cannot scale to model languages with higher-order store or dynamic type-tag generation. Since these features are common in dynamically-typed programming languages, it makes sense that a good semantic foundation for gradual typing should include support for these features.

We were therefore motivated to seek an alternative approach that features the best of both worlds; this is the topic of my subsequent project.

3 Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory

3.1 Guarded Type Theory

In this work, we seek to define a model of gradual typing that combines the expressive nature of step-indexing with the compositional nature of denotational semantics. A key ingredient in achieving this goal is *guarded type theory*, which is a non-standard foundations with axioms that capture the reasoning principles of step-indexing [4, 3]. Guarded type theory is a kind of metalanguage in which we can define denotational semantics of programming languages involving general recursion. Guarded type theory differs from classical domain theory in that it can scale to arbitrarily complex recursive definitions.

The new reasoning principles provided by guarded type theory are as follows. First, there is an operation $\triangleright$ (“later”) that acts on types. A value of type $\triangleright X$ intuitively represents a value of type X available one “computational step” in the future. There is a function `next` that delays an element available now to be available later. Lastly, there is *guarded recursion*. This principle states that there is a unique solution to any recursive type equation of the form $X \cong F(\triangleright X)$, that is, any recursive type equation where the variable X is guarded by $\triangleright$.

The main fact to remember about guarded type theory is that, as with step-indexing, it is *intensional*, or step-aware: the denotation of a term in guarded type theory reflects the computational steps it takes. This is in contrast to classical domain theory, where the steps taken by a term are not reflected in its denotation. The intensional nature of guarded type theory makes defining a semantics easier in some places, while introducing challenges in others.

3.2 Result

Our first major result is a denotational semantics for a simple gradually-typed language defined using guarded type theory. The language we model has natural numbers, products, and call-by-value-style functions. We will introduce more details about the language as they are needed. (We refer the reader to our POPL 2025 paper for the full definitions [9].) Recall from Section 1.2 that any semantics for gradually-typed languages must model errors and divergence. Therefore, our semantics is a partial function from closed terms of type `Nat` to either a natural number or a runtime error:

$$- \Downarrow: \{M \mid \vdash M : \text{Nat}\} \rightarrow \mathbb{N} \cup \{\mathcal{U}\}$$

It is a partial function because the program may not terminate. Here, $\mathcal{U}$ is notation for a runtime type error. We write $M \Downarrow n$ and $M \Downarrow \mathcal{U}$ to mean this semantics is defined as a number or error, and $M \Uparrow$ to mean the semantics is undefined, representing divergence.

Our semantics satisfies the two key properties discussed in Section 1.1. First, it validates **type-based reasoning**. That is, if two closed terms of type `Nat` are equivalent according to the equational theory of the language, then they have the same behavior: either they both evaluate to the same natural number n , they both error, or they both fail to terminate. Second, our semantics satisfies the **graduality** property: If M and N are closed terms of type `Nat` where M is more precise than N , then either M errors, in which case we don’t care what N does, or M and N have the same behavior. These results are stated in detail below.

Theorem 3.1 (Type-Based Reasoning). *Let $\cdot \vdash M, N : \text{Nat}$. If $M \equiv N$, then M and N have the same*

behavior, in that:

$$\begin{aligned} M \Downarrow n &\iff N \Downarrow n \\ M \Downarrow \mathcal{U} &\iff N \Downarrow \mathcal{U} \\ M \Uparrow &\iff N \Uparrow \end{aligned}$$

Theorem 3.2 (Graduality). *Let $\cdot \vdash M, N : \text{Nat}$. If $M \sqsubseteq N$, then either:*

1. $M \Downarrow \mathcal{U}$.
2. $M \Uparrow$ and $N \Uparrow$.
3. $M \Downarrow n$ and $N \Downarrow n$ for some natural number n .

In the remainder of this section, we will discuss the main ideas in the construction of the semantics.

3.3 Modeling Gradually-Typed Programs

We now dive into more detail about our semantics, with an eye towards understanding the places where guarded type theory makes defining the semantics simple, and the places where it introduces challenges. The first question we need to address is how to model effectful programs. Recall that the language we are modeling involves errors and computational steps. Given a type A , we define a new type we call the *free error domain* on A , written $L_{\mathcal{U}}A$. We define the free error domain on A via guarded recursion as the solution to the equation

$$L_{\mathcal{U}}A \cong A + 1 + \triangleright L_{\mathcal{U}}A.$$

That is, an element of the free error domain is either a returned value of type A , an error, or an element defined “later”. The latter can be thought of as a “thinking” or “computing” element, such that after one time step has passed we will be able to access the element. Of course, this element may itself be a value, an error, or another thinking element.

We will use the notation $\eta : A \rightarrow L_{\mathcal{U}}A$, $\mathcal{U} : L_{\mathcal{U}}A$, and $\theta : \triangleright L_{\mathcal{U}}A \rightarrow L_{\mathcal{U}}A$ to refer to the three injections into the sum.

3.4 Modeling Dynamic Typing

Next, we consider how to model dynamic typing. Recall that our language, like all gradually-typed languages, has a special *dynamic type* written $?$ (pronounced “dyn”). The type $?$ represents a statically-unknown type. The language we are modeling has natural numbers, products, and call-by-value-style functions. Thus, to model the dynamic type, we seek a semantic type D satisfying the following equation.

$$D \cong \text{Nat} + (D \times D) + (D \rightarrow L_{\mathcal{U}}D).$$

Notice that D occurs on the left of the arrow in the function case; this means that the equation does not have inductive or coinductive solutions. It *does* have solutions using classical domain theory, but in this work we are using guarded type theory rather than classical domain theory. Indeed, consider the related equation where we guard the negative occurrence by placing a later operator outside the function case:

$$D \cong \text{Nat} + (D \times D) + \triangleright (D \rightarrow L_{\mathcal{U}}D).$$

We can solve above equation using a combination of least fixpoint and guarded recursion. More details on solving the equation are given in our POPL 2025 paper [9].

$$\begin{array}{cccc}
\frac{M \sqsubseteq N : c}{\text{up } c M \sqsubseteq N : A'} \text{ UPL} & \frac{M \sqsubseteq N : A}{M \sqsubseteq \text{up } c N : c} \text{ UPR} & \frac{M \sqsubseteq N : A'}{\text{dn } c M \sqsubseteq N : c} \text{ DNL} & \frac{M \sqsubseteq N : c}{M \sqsubseteq \text{dn } c N : A} \text{ DNR}
\end{array}$$

Figure 1: Cast Axioms (Simplified Form)

Though this use of the later modality gives us a semantics for the dynamic type essentially for free, it comes at a cost: **A program must take an observable step whenever it wants to inspect a value of the dynamic type that happens to be a function.** This has major implications for the relational semantics, as we discuss below.

3.5 Relational Semantics

Before moving forward, we discuss some additional important concepts in gradual typing. Recall the notion of term precision introduced in Section 1.1. There is an analogous ordering on types called *type precision* that represents when one type is more static or dynamic than another. Specifically, we write $A \sqsubseteq A'$ to represent that type A is more precise (i.e., more static) than type A' . We use an explicit syntax of *type precision derivations* $c : A \sqsubseteq A'$; see our POPL 2025 paper for the details [9]. The dynamic type $?$ is the top element in the type precision ordering, as it is the *least* precise type.

Gradually-typed languages are typically elaborated to a lower-level representation called a *cast calculus*. This involves inserting explicit coercion terms called *casts* at the boundaries between static and dynamically-typed code to ensure that the static types are enforced at runtime. Specifically, whenever $c : A \sqsubseteq A'$, there is an *upcast* $\text{up } c$ from terms A to A' , and a *downcast* $\text{dn } c$ from A' to A . Upcasts always succeed, but downcasts can fail at runtime if the types are incompatible. The key point to remember about casts in our semantics is that **casts can take steps**. For example, downcasting the dynamic type to a function type involves a step since the function is only available later.

Prior work has established that the graduality property essentially boils down to the behavior of the cast terms [12], shown in Figure 1. There are four key axioms that capture the intended behavior of casts, two for upcasts and two for downcasts. These axioms describe the intended relationship between a type precision and the corresponding up/downcasts. The basic idea is that **upcasts make terms less precise**, while **downcasts make terms more precise**.

The axioms shown here are simplified from the original formulation as used in prior work. However, compositional reasoning allows us to *derive* the more complicated rules from the ones shown here. The alternative would be to prove the more complicated ones from scratch in the model as was done in prior operational approaches such as our Gradual Typing for Effect Handlers paper [13]. The derivation of the original cast axioms from the simplified ones serves as a specific example of the benefits of a compositional approach to the semantics of gradual typing.

3.5.1 A First Attempt at a Relational Model

We next examine how we might attempt to model type and term precision. We will base the relational semantics on the prior denotational model of New and Licata [16], as follows:

- We model our types as sets with a partial ordering relation, representing the term precision ordering on values.

- Terms will denote monotone functions into the free error domain.
- A type precision relationship $c : A \sqsubseteq A'$ will be modeled as a *representable relation* between the denoted posets. The precise definition of representability is not important for this discussion; what is important is that it allows us to interpret the four cast axioms discussed in the previous section.
- Term precision will be modeled as a logical-relations style relationship between the denoted monotone functions, that is, related inputs are mapped to related outputs.

Unfortunately, the New-Licata semantics does not carry over directly to the guarded setting. The issue lies in the requirement that the relations be representable. In more detail, the problem is a consequence of the tension between the requirements of *step-insensitivity* and *transitivity*, which are both necessary properties for our semantics. A relation is step-insensitive when it is oblivious to the steps taken by terms. We need our relations to be step-insensitive because on the one hand casts can take steps, but on the other hand representability requires that casts be related to the identity function, which does not step. For example, recall the DNR rule:

$$\frac{M \sqsubseteq N : c}{M \sqsubseteq \text{dn } c \, N : A} \text{ DNR}$$

Notice that the downcast may take an observable step beyond N , but M remains the same in the premise and conclusion of the rule. In other words, M may become “out of sync” with the downcast.

In addition to step-insensitivity, we also need transitivity. Transitivity is necessary for us to be able to compose smaller proofs into larger proofs, e.g., deriving the original cast axioms from the simplified ones relies on transitivity. However, as we prove in the paper via a no-go theorem, **unrestricted step-insensitivity is not compatible with transitivity**: it is not possible to define a relation that has both properties.

3.5.2 Adjusting the Model: Perturbations

Transitivity is non-negotiable given our goal of a compositional semantics, so we choose to give up on (unrestricted) step-insensitivity. Specifically, we work with a *lock-step ordering relation* that is transitive but *sensitive to steps*. This breaks the representability condition, which as we saw requires step-insensitivity. (Casts may step and so are not necessarily in lock-step with the identity function.) To remedy this situation, we restore a small amount of step-insensitivity in a controlled manner using special synchronizing terms we call *perturbations*. These are explicitly inserted delay terms that put both sides into lock-step. Perturbations mimic the stepping behavior of casts, but are *weakly bisimilar* to the identity function, that is, equivalent to the identity function when ignoring steps. Finally, we relax the definition of representability to allow for casts to be lock-step related to perturbations rather than only to the identity function. We call this notion *quasi-representability*.

The updated axioms for casts are as shown in Figure 2. Notice that on the sides opposite the casts, we allow for there to be a *perturbation*. The perturbation mimics the stepping behavior of the cast, allowing both sides to remain synchronized.

3.6 Summary of the Model

Having introduced perturbations and quasi-representable relations, we now summarize the final version of the semantics, with the new features [highlighted in blue](#).

- Types denote sets with:

$$\begin{array}{c}
\frac{M \sqsubseteq N : c}{\text{up } c \, M \sqsubseteq \delta^{r,e}(N) : A'} \text{ UPL} \qquad \frac{M \sqsubseteq N : A}{\delta^{l,e}(M) \sqsubseteq \text{up } c \, N : c} \text{ UPR} \\
\\
\frac{M \sqsubseteq N : A'}{\text{dn } c \, M \sqsubseteq \delta^{r,p}(N) : c} \text{ DNL} \qquad \frac{M \sqsubseteq N : c}{\delta^{l,p}(M) \sqsubseteq \text{dn } c \, N : A} \text{ DNR}
\end{array}$$

Figure 2: Adjusted Cast Axioms

1. A partial ordering relation (terms must be in lock-step).
 2. A weak bisimilarity relation.
 3. A monoid of syntactic perturbations.
- Terms denote *monotone*, bisimilarity-preserving functions into the free error domain.
 - Type precision denotes quasi-representable relations that interact with syntactic perturbations.
 - Term precision is modeled by taking the lock-step ordering on the denoted monotone functions, and closing it under weak bisimilarity on both sides.

The last point deserves elaboration: We define the denotation of term precision $M \sqsubseteq N$ to mean that there are functions f and g such that $M \approx f \sqsubseteq g \approx N$ where $\approx$ is weak bisimilarity and $\sqsubseteq$ is the lock-step ordering.

The important takeaway here is that we have separated term precision into a combination of weak bisimilarity and lock-step reasoning, using syntactic perturbations as a means to explicitly synchronize terms where needed, which allows for compositional reasoning while still accommodating the steps taken by casts.

3.7 Agda Formalization

In parallel with developing the theory discussed in this paper, we have developed a partial formalization of our results in Guarded Cubical Agda [24], which extends the Agda type checker with support for the primitives of guarded type theory. The formalization is available online [8]. Most of the model constructions have been formalized, with the exception of some lemmas involving quasi-representable relations. We also did not formalize the translation from the syntax into our semantic model because it led to prohibitively poor performance of typechecking in Agda.

4 Extending to More Advanced Language Features

Our next major step is to extend our semantics to accommodate languages with higher-order store and/or dynamic type-tag generation. We plan to begin with higher-order store, and then depending on the difficulty we may also consider dynamic type-tag generation as part of this thesis.

Higher-order store allows for the storage of values of arbitrary type, including functions. Concretely, there

is a type of *references* of type A , denoted $\text{ref } A$. The programmer can allocate a new reference cell of type A , and given a reference cell the programmer can perform get and set operations to read and write the value in the cell.

For the next step of my thesis work, I will consider the language obtained by augmenting the simple gradually-typed language we modeled previously with the features of higher-order store. My goal is to adapt our guarded model of gradual typing to this language. The main technical change that this will require is the move from a “set-based” semantics to a semantics based in *presheaves* over a category of worlds. This is needed to account for the fact that shape of the store may change over the course of the program as the programmer allocates new reference cells. The world tracks the reference cells that have been allocated, and a presheaf on worlds is essentially a set that is allowed to vary based on the current world, i.e., the current shape of the store. The main challenge will be to adapt the constructs of our guarded model to the setting of presheaves. I plan to draw guidance from the work of Sterling et al. [21], who give a denotational semantics for higher-order store using guarded type theory.

5 Plan for Remainder of Thesis Work

Below is an approximate timeline I am proposing for completing the remainder of the work. This is subject to change based on the work that we agree upon.

- May 2025 - October 2025: complete the majority of the remaining technical work.
- October 2025 - January 2026: wrap-up technical work and focus on writing the dissertation.
- January 2026 - April 2026: complete a first draft of the dissertation; begin preparing for defense.
- May 2026 or August 2026 - defend thesis and complete edits to dissertation.
- July 2026 - submit the new technical work to the POPL conference.

Note: I plan to be a GSI (a graduate teaching assistant) during the Fall 2025 semester. Therefore, the time I will have to dedicate to the dissertation will be reduced during this period.

References

- [1] Amal J. Ahmed. Step-indexed syntactic logical relations for recursive and quantified types. In *15th European Symposium on Programming, ESOP 2006, Vienna, Austria*, volume 3924, pages 69–83, March 2006.
- [2] Andrew W. Appel and David McAllester. An indexed model of recursive types for foundational proof-carrying code. *ACM Transactions on Programming Languages and Systems*, 23(5):657–683, September 2001.
- [3] Magnus Baunsgaard Kristensen, Rasmus Ejlers Mogelberg, and Andrea Vezzosi. Greatest hits: Higher inductive types in coinductive definitions via induction under clocks. In *ACM/IEEE Symposium on Logic in Computer Science (LICS), Haifa, Israel*, 2022.
- [4] Lars Birkedal, Rasmus Ejlers Mogelberg, Jan Schwinghammer, and Kristian Stovring. First steps in synthetic guarded domain theory: Step-indexing in the topos of trees. In *ACM/IEEE Symposium on Logic in Computer Science (LICS), Toronto, Canada*, 2011.

- [5] Lars Birkedal, Kristian Støvring, and Jacob Thamsborg. Realizability semantics of parametric polymorphism, general references, and recursive types. In *Foundations of Software Science and Computational Structures, Berlin*, pages 456–470, 2009.
- [6] Matteo Cimini and Jeremy G. Siek. The gradualizer: A methodology and algorithm for generating gradual type systems. In *ACM Symposium on Principles of Programming Languages (POPL)*, St. Petersburg, Florida, 2016.
- [7] Ronald Garcia, Alison M. Clark, and Éric Tanter. Abstracting gradual typing. In *ACM Symposium on Principles of Programming Languages (POPL)*, St. Petersburg, Florida, pages 429–442, 2016.
- [8] Eric Giovannini, Tingting Ding, and Max S. New. Agda Formalization for "Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory", October 2024.
- [9] Eric Giovannini, Tingting Ding, and Max S. New. Denotational semantics of gradual typing using synthetic guarded domain theory. *Proc. ACM Program. Lang.*, 9(POPL), January 2025.
- [10] Meven Lennon-Bertrand, Kenji Maillard, Nicolas Tabareau, and Éric Tanter. Gradualizing the calculus of inductive constructions. *ACM Trans. Program. Lang. Syst.*, 44(2), apr 2022.
- [11] Georg Neis, Derek Dreyer, and Andreas Rossberg. Non-parametric parametricity. In *International Conference on Functional Programming (ICFP)*, Edinburgh, Scotland, pages 135–148, September 2009.
- [12] Max S. New and Amal Ahmed. Graduality from embedding-projection pairs. *Proceedings of the ACM on Programming Languages (PACMPL)*, 2(ICFP), jul 2018.
- [13] Max S. New, Eric Giovannini, and Daniel R. Licata. Gradual typing for effect handlers. *Proceedings of the ACM on Programming Languages (PACMPL)*, 7(OOPSLA2):1758–1786, 2023.
- [14] Max S. New, Eric Giovannini, and Daniel R. Licata. Gradual typing for effect handlers, 2023.
- [15] Max S. New, Dustin Jamner, and Amal Ahmed. Graduality and parametricity: Together again for the first time. *Proceedings of the ACM on Programming Languages (PACMPL)*, 4(POPL), jan 2020.
- [16] Max S. New and Daniel R. Licata. Call-by-name gradual type theory. In *Formal Structures for Computation and Deduction*, Oxford England, 2018.
- [17] Max S. New, Daniel R. Licata, and Amal Ahmed. Gradual type theory. *Proceedings of the ACM on Programming Languages (PACMPL)*, 3(POPL), jan 2019.
- [18] Jeremy G. Siek and Tianyu Chen. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *J. Funct. Program.*, 31:e30, 2021.
- [19] Jeremy G. Siek and Walid Taha. Gradual typing for functional languages. In *Scheme and Functional Programming Workshop (Scheme)*, pages 81–92, September 2006.
- [20] Jeremy G. Siek, Michael M. Vitousek, Matteo Cimini, and John Tang Boyland. Refined Criteria for Gradual Typing. In *1st Summit on Advances in Programming Languages (SNAPL 2015)*, volume 32, pages 274–293, 2015.
- [21] Jonathan Sterling, Daniel Gratzer, and Lars Birkedal. Denotational semantics of general store and polymorphism. *CoRR*, 2022.
- [22] Sam Tobin-Hochstadt and Matthias Felleisen. Interlanguage migration: From scripts to programs. In *Dynamic Languages Symposium (DLS)*, pages 964–974, October 2006.

- [23] Sam Tobin-Hochstadt and Matthias Felleisen. The design and implementation of typed scheme. In *ACM Symposium on Principles of Programming Languages (POPL)*, San Francisco, California, 2008.
- [24] Niccolò Veltri and Andrea Vezzosi. Formalizing π -calculus in guarded cubical agda. In *ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 270–283, 2020.